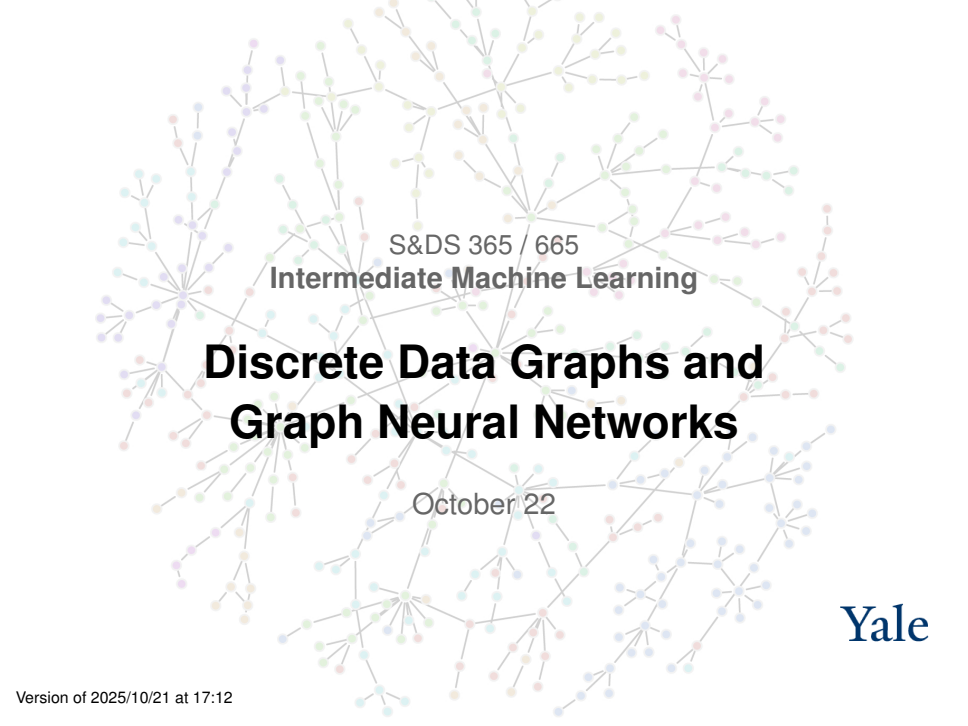




S&DS 365 / 665
Intermediate Machine Learning

Discrete Data Graphs and Graph Neural Networks

October 22

Yale

Friendly reminders

- Assignment 3 out; due next Thursday
- Assignment 4 posted next week
- Midterm scores and mid-semester grades posted

For today

- Quick recap
- Graphs for discrete data
- Intro to graph neural networks



Graphs

- A natural language for describing various data
- Give information about relationships between variables
- Associated with each multivariate distribution

Markov Property

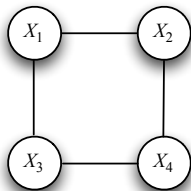
A probability distribution P satisfies the *global Markov property* with respect to a graph G if:

for any disjoint vertex subsets A , B , and C such that C separates A and B ,

$$X_A \perp\!\!\!\perp X_B \mid X_C.$$

- X_A are the random variables X_j with $j \in A$.
- C separates A and B means that there is no path from A to B that does not pass through C .

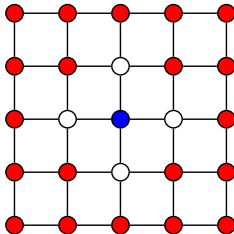
Example



$$\begin{array}{lcl} X_1 & \perp\!\!\!\perp & X_4 \quad | \quad X_2, X_3 \\ X_2 & \perp\!\!\!\perp & X_3 \quad | \quad X_1, X_4 \end{array}$$

Example: 2-dimensional grid

The blue node is independent of the red nodes given the white nodes.



Special case

If $(i, j) \notin E$ then

$$X_i \perp\!\!\!\perp X_j \mid \{X_k : k \neq i, j\}$$

$$A = \{i\}, B = \{j\}, C = \{k \neq i, j\}$$

Special case

If $(i, j) \notin E$ then

$$X_i \perp\!\!\!\perp X_j \mid \{X_k : k \neq i, j\}$$

$$A = \{i\}, B = \{j\}, C = \{k \neq i, j\}$$

Lack of an edge from i to j implies that X_i and X_j are independent given all of the other random variables.

Graph estimation

- A graph G represents the class of distributions, $\mathcal{P}(G)$, the distributions that are Markov with respect to G
- Graph estimation: Given n samples $X_1, \dots, X_n \sim P$, estimate the graph G .

Factored form

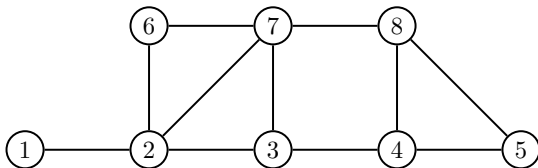
Theorem (Hammersley, Clifford, Besag)

A positive distribution over random variables $X_1, \dots, X_p$ satisfies the Markov properties of graph G if and only if it can be represented as

$$p(X) \propto \prod_{c \in \mathcal{C}} \psi_c(X_c)$$

where $\mathcal{C}$ is the set of cliques in the graph G .

Example



$$P(X) \propto \psi_{12}(X_1, X_2) \cdot \psi_{267}(X_2, X_6, X_7) \cdot \psi_{237}(X_2, X_3, X_7) \cdot \\ \psi_{34}(X_3, X_4) \cdot \psi_{78}(X_7, X_8) \cdot \psi_{458}(X_4, X_5, X_8)$$

Gaussian case

Let $\Omega = \Sigma^{-1}$ be the precision matrix.

A zero in Ω indicates a *lack of the corresponding edge* in the graph

So, the adjacency matrix of the graph is

$$A = (\mathbb{1}(\Omega_{ij} \neq 0))$$

That is,

$$A_{ij} = \begin{cases} 1 & \text{if } |\Omega_{ij}| > 0 \\ 0 & \text{otherwise} \end{cases}$$

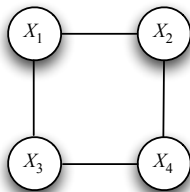
Gaussian case

$$\Omega \equiv \Sigma^{-1} = \begin{pmatrix} * & * & 0 \\ * & * & * \\ 0 & * & * \end{pmatrix}$$



Gaussian case

$$\Omega \equiv \Sigma^{-1} = \begin{pmatrix} * & * & * & 0 \\ * & * & 0 & * \\ * & 0 & * & * \\ 0 & * & * & * \end{pmatrix}$$



$$X_1 \perp\!\!\!\perp X_4 \mid X_2, X_3$$



Discrete Graphical Models

Challenges of handling discrete data:

- Models don't have closed form; can't compute normalizing constant
- Need to use Gibbs sampling, variational inference
- *No analogue of the graphical lasso*

Discrete Graphical Models

- Positive distributions can be represented by an exponential family,

$$p(X) \propto \exp \left(\sum_{c \in \mathcal{C}} \phi_c(X_c) \right)$$

where $\phi_c = \log \psi_c$ from Hammersley-Clifford

- Special case: Ising Model (discrete Gaussian)

$$p_{\beta}(X) \propto \exp \left(\sum_{i \in V} \beta_i X_i + \sum_{(i,j) \in E} \beta_{ij} X_i X_j \right).$$

From edges to cliques

Take $\beta_i \equiv 0$ for simplicity

If we have a triangle (i, j, k) in the graph then the potential function corresponds to

$$\begin{aligned}\psi_{ijk}(X_i, X_j, X_k) &= e^{\beta_{ij} X_i X_j} \cdot e^{\beta_{jk} X_j X_k} \cdot e^{\beta_{ik} X_i X_k} \\ &= e^{\beta_{ij} X_i X_j + \beta_{jk} X_j X_k + \beta_{ik} X_i X_k}\end{aligned}$$

Ising

We have a graph with edges E and vertices V . Each node i has a random variable X_i that can be “up” ($X_i = 1$) or “down” ($X_i = 0$)

$$p_{\beta}(x_1, \dots, x_n) \propto \exp \left(\sum_{s \in V} \beta_s x_s + \sum_{(s,t) \in E} \beta_{st} x_s x_t \right)$$

Since $2X_i - 1 \in \{-1, 1\}$ if $X_i \in \{0, 1\}$, can re-parameterize model using sample space $X_i = \pm 1$.

Ising

We have a graph with edges E and vertices V . Each node i has a random variable X_i that can be “up” ($X_i = 1$) or “down” ($X_i = 0$)

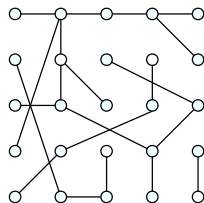
$$p_{\beta}(x_1, \dots, x_n) \propto \exp \left(\sum_{s \in V} \beta_s x_s + \sum_{(s,t) \in E} \beta_{st} x_s x_t \right)$$

E are the set of edges, V are the vertices. Imagine the Z_i are votes of politicians, and the edges encode the social network of party affiliations

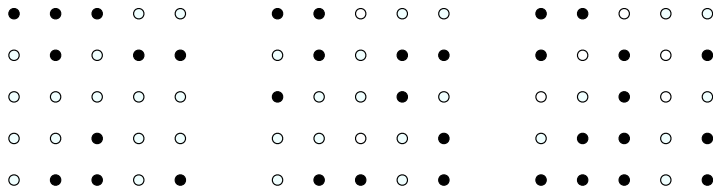
Since $2X_i - 1 \in \{-1, 1\}$ if $X_i \in \{0, 1\}$, can re-parameterize model using sample space $X_i = \pm 1$.

Graph Estimation

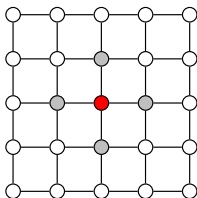
- Given n i.i.d. samples from an Ising distribution, $\{X_i, i = 1, \dots, n\}$, (each is a p -vector of $\{0, 1\}$ values) identify underlying graph



- Multiple examples are observed:



Local Distributions



- Consider Ising model $p_{\beta}(X) \propto \exp \left(\sum_{i \in V} \beta_i X_i + \sum_{(i,j) \in E} \beta_{ij} X_i X_j \right)$.
- Conditioned on $(x_2, \dots, x_p)$, variable $X_1 \in \{0, 1\}$ has probability mass function given by a logistic function,

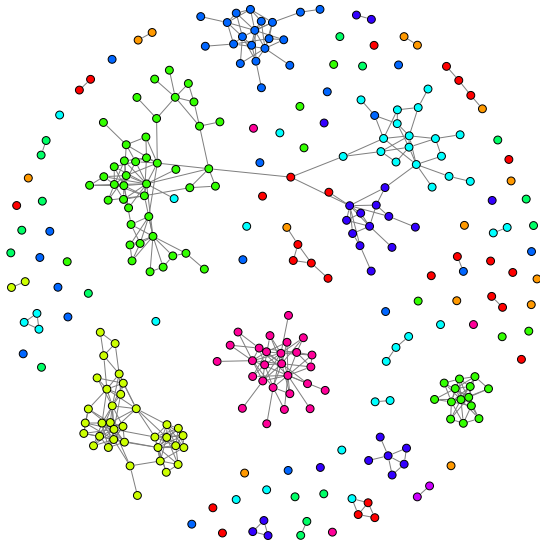
$$p(X_1 = 1 \mid x_2, \dots, x_p) = \text{sigmoid} \left(\beta_1 + \sum_{j \in \mathcal{N}(1)} \beta_{1j} x_j \right)$$

Parallel lasso (sparse logistic regressions)

Strategy

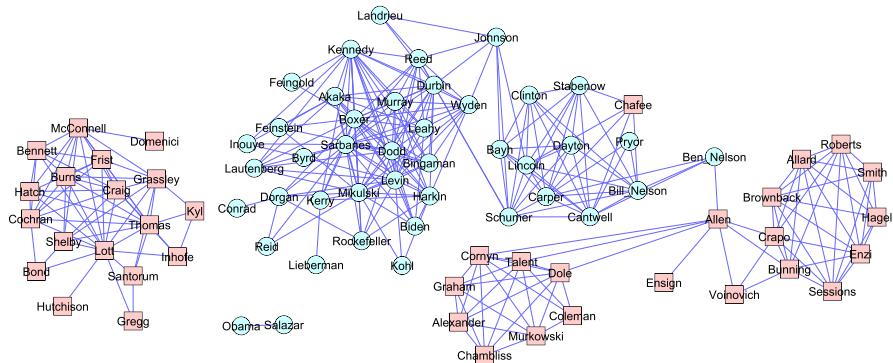
- Perform ℓ_1 regularized logistic regression of each node X_i on $X_{\setminus i} = \{X_j, j \neq i\}$ to estimate neighbors $\hat{\mathcal{N}}(i)$
- Two versions:
 - ▶ Create an edge (i, j) if $j \in \hat{\mathcal{N}}(i)$ **and** $i \in \hat{\mathcal{N}}(j)$
 - ▶ Create an edge (i, j) if $j \in \hat{\mathcal{N}}(i)$ **or** $i \in \hat{\mathcal{N}}(j)$

S&P 500: Ising Model (Price up or down?)



Voting Data

Voting records of US Senate, 2006-2008



Scaling behavior: Performance with data size

Maximum degree d of the p variables. Sample size n must satisfy

$$\text{Ising model: } n \geq d^3 \log p$$

$$\text{Graphical lasso: } n \geq d^2 \log p$$

$$\text{Parallel lasso: } n \geq d \log p$$

$$\text{Lower bound: } n \geq d \log p$$

- Each method makes different *incoherence assumptions*:
 - ▶ Correlations between unrelated variables not too large

Summary: Graphs for discrete data

- A positive distribution factors into product of potential functions on the cliques of the graph
- Graphs and independence relations are same for discrete data
- Ising models are discrete Gaussians
- No version of the graphical lasso holds for discrete data; instead, we use the parallel lasso

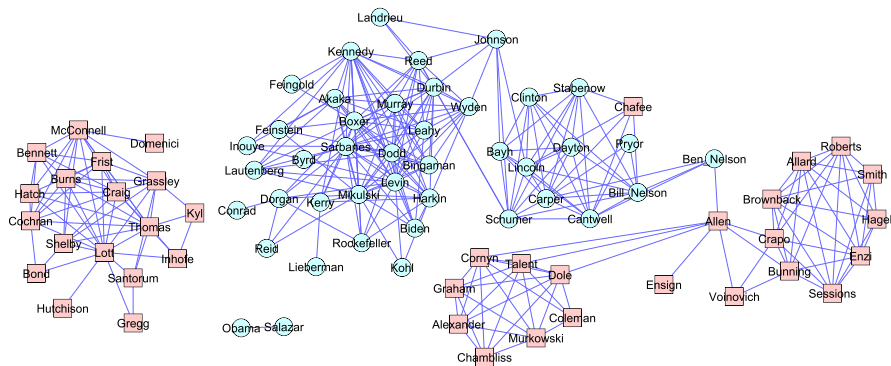
Graph neural networks

Next, we'll discuss graph neural networks, following this article:

<https://distill.pub/2021/understanding-gnns/>

Recall: Voting Data

Voting records of US Senate, 2006-2008

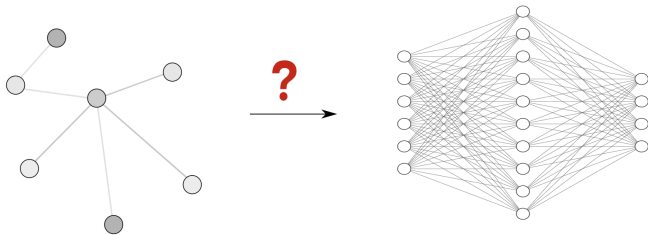


Suppose we have the graph and other covariates for each node or edge.
How can we classify the senators according to political orientation?

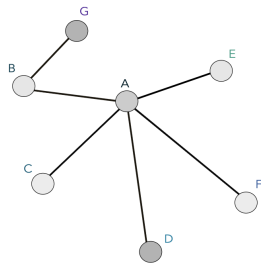
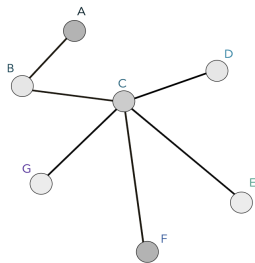
Graph neural networks

- We'll discuss how GNNs correspond to CNNs
- The graph Laplacian plays a central role

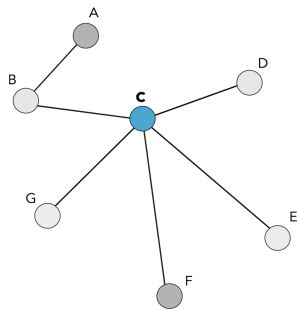
Equivariance problem



Equivariance problem



Graph Laplacian



Input Graph G

	A	B	C	D	E	F	G
A	1	-1					
B	-1	2	-1				
C		-1	5	-1	-1	-1	-1
D			-1	1			
E			-1		1		
F			-1			1	
G			-1				1

Laplacian L of G

Polynomials of the Laplacian

$$p_w(L) = w_0 I_n + w_1 L + w_2 L^2 + \cdots w_d L^d$$

If $\text{dist}(u, v) > i$ then the (u, v) entry of L^i is zero

- This is analogous to a CNN filter (kernel)
- The weights w_i play role of filter coefficients
- Degree d of polynomial plays role of the size of the kernel

The Laplacian is a Mercer kernel

- Symmetric $L_{uv} = L_{vu}$
- Positive-definite:

$$f^T L f = \text{trace}(L f f^T) = \sum_{(u,v) \in E} (f_u - f_v)^2 \geq 0$$

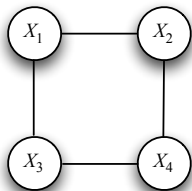
Classical Laplacian

$$\Delta f = \text{trace} \left(\frac{\partial^2 f}{\partial x_i \partial x_j} \right) = \frac{\partial^2 f}{\partial x_1^2} + \cdots + \frac{\partial^2 f}{\partial x_n^2}$$

The Laplace operator in its various manifestations is the most beautiful and central object in all of mathematics. Probability theory, mathematical physics, Fourier analysis, partial differential equations, the theory of Lie groups, and differential geometry all revolve around this sun, and its light even penetrates such obscure regions as number theory and algebraic geometry.

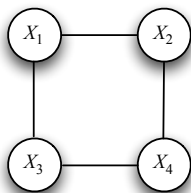
Edward Nelson, *Tensor Analysis*

Sanity checks



What is the Laplacian L ?

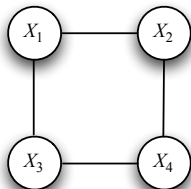
Sanity checks



What is the Laplacian L ?

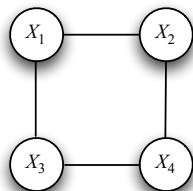
$$L = \begin{pmatrix} 2 & -1 & -1 & 0 \\ -1 & 2 & 0 & -1 \\ -1 & 0 & 2 & -1 \\ 0 & -1 & -1 & 2 \end{pmatrix}$$

Sanity checks



What is L^2 ?

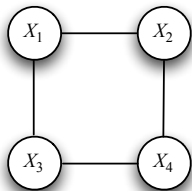
Sanity checks



What is L^2 ?

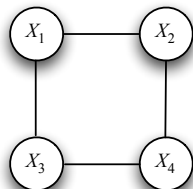
$$L^2 = \begin{pmatrix} 6 & -4 & -4 & 2 \\ -4 & 6 & 2 & -4 \\ -4 & 2 & 6 & -4 \\ 2 & -4 & -4 & 6 \end{pmatrix}$$

Sanity checks



If $x = (1, 2, 3, 4)^T$ what is $h = \text{ReLU}(Lx)$?

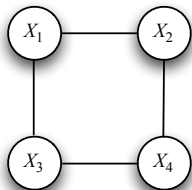
Sanity checks



If $x = (1, 2, 3, 4)^T$ what is $h = \text{ReLU}(Lx)$?

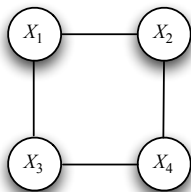
$$\text{ReLU}(Lx) = \text{ReLU}((-3, -1, 1, 3)^T) = (0, 0, 1, 3)^T$$

Sanity checks



If $x = (1, 2, 3, 4)^T$ what is $x^T L x$?

Sanity checks



If $x = (1, 2, 3, 4)^T$ what is $x^T L x$?

$$x^T L x = \sum_{(u,v) \in E} (x_u - x_v)^2 = 10$$

Equivariance

If we compute a feature map f , we'd like it to not depend on a reordering P of the nodes.

The reordering

$$1 \leftarrow 2$$

$$2 \leftarrow 3$$

$$3 \leftarrow 1$$

corresponds to permutation matrix

$$P = \begin{pmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{pmatrix}$$

Whence equivariance

A transformation $f : \mathbb{R}^n \longrightarrow \mathbb{R}^n$ is equivariant if

$$f(Px) = Pf(x)$$

for any permutation matrix P , where $PP^T = I$.

The transformed data and Laplacian are

$$x \longrightarrow Px$$

$$L \longrightarrow PLP^T$$

$$L^i \longrightarrow PL^iP^T$$

Whence equivariance

A transformation $f : \mathbb{R}^n \rightarrow \mathbb{R}^n$ is equivariant if

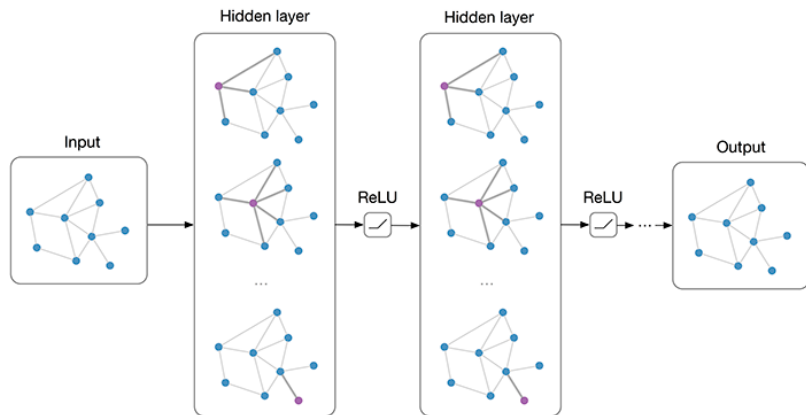
$$f(Px) = Pf(x)$$

for any permutation matrix P , where $PP^T = I$.

The transformed polynomial kernels are

$$\begin{aligned} f(Px) &= \sum_{i=0}^d w_i (PL^i P^T) Px \\ &= \sum_{i=0}^d w_i PL^i x \\ &= P \sum_{i=0}^d w_i L^i x \\ &= Pf(x) \end{aligned}$$

Building layers



Building layers

- For example, each hidden layer gives three “feature maps”

$$h_1 = \text{relu}(p_{w_1}(L)x)$$

$$h_2 = \text{relu}(p_{w_2}(L)x)$$

$$h_3 = \text{relu}(p_{w_3}(L)x)$$

where each $h_i \in \mathbb{R}^{|V|}$ with $|V| = 9$.

- The final output is then computed as

$$y = \beta_1 h_1 + \beta_2 h_2 + \beta_3 h_3$$

so that the output $y \in \mathbb{R}^{|V|}$ is a vector

Summary: Graph neural nets

- Certain data have natural graphical structure
- GNNs are analogues of CNNs for graphs
- Based on use of graph Laplacian
- Independent of ordering of nodes (equivariant)
- We'll go deeper on Assn 4!

A recent framework that combines ideas from CNNs and GNNs

 Cornell University

We gratefully acknowledge support from
member institutions

arXiv > cs > arXiv:2310.03240

Search...
Help | Advanced Search

Computer Science > Machine Learning

[Submitted on 5 Oct 2023]

Relational Convolutional Networks: A framework for learning representations of hierarchical relations

Awni Altabaa, John Lafferty

A maturing area of research in deep learning is the development of architectures that can learn explicit representations of relational features. In this paper, we focus on the problem of learning representations of hierarchical relations, proposing an architectural framework we call "relational convolutional networks". Given a sequence of objects, a "multi-dimensional inner product relation" module produces a relation tensor describing all pairwise relations. A "relational convolution" layer then transforms the relation tensor into a sequence of new objects, each describing the relations within some group of objects at the previous layer. Graphlet filters, analogous to filters in convolutional neural networks, represent a template of relations against which the relation tensor is compared at each grouping. Repeating this yields representations of higher-order, hierarchical relations. We present the motivation and details of the architecture, together with a set of experiments to demonstrate how relational convolutional networks can provide an effective framework for modeling relational tasks that have hierarchical structure.

Comments: 18 pages, 7 figures, 4 tables

Subjects: Machine Learning (cs.LG)

Next topic: Reinforcement learning